

Introduction to Java

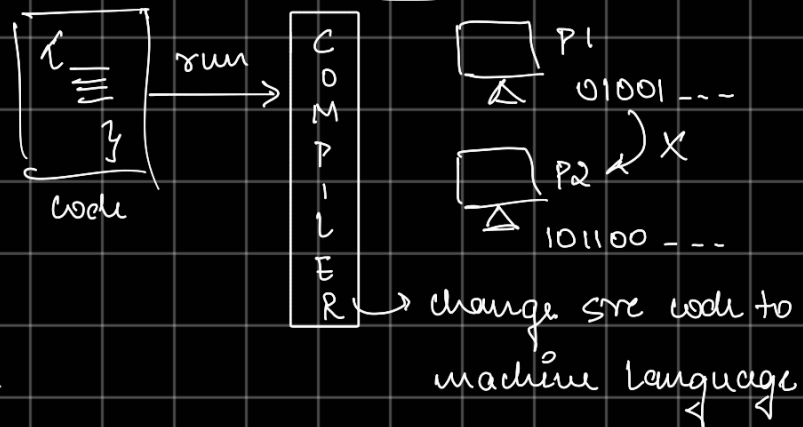
- 1980/90s → C/C++ [fast, simple, low-level, close to hardware, direct interact w processor]

⇒ Why Java?

↳ problem with C/C++ → portability

- compiler ne jis code ko compile karta hai, woh kisi ek platform ke liye karta hoga.

eg machine code on P1 won't work on P2; if want to run on P2, then again have to compile using compiler of P2

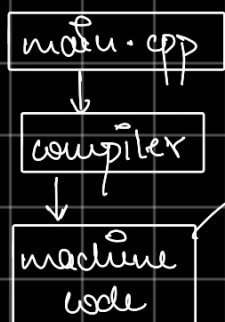
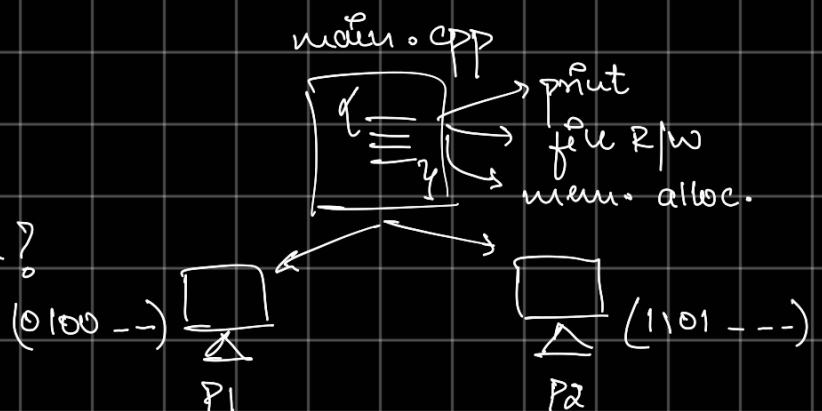


↳ Platform (C/C++ → platform dependent)
↳ processor
↳ OS

* whenever to run a src code on a platform, first have to compile the code for that particular platform
→ problem of portability

- Intel x86 + windows → P1
- ARM + mac → P2

* Why different binary generated?



OS

system
libraries

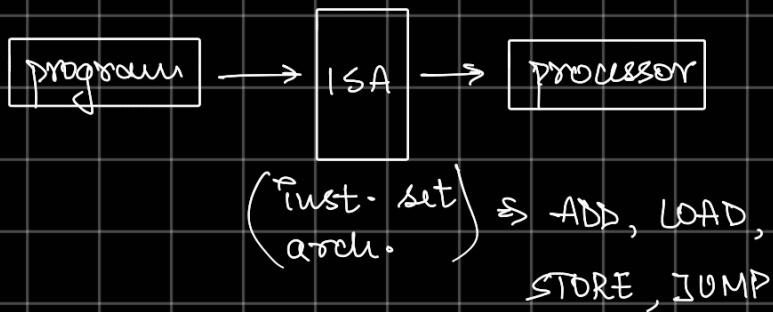
are called

to perform commands

different for different OS ⇒ diff. binary code

- processor (CPU) $\rightarrow$ billions / trillions of transistors $\rightarrow$ ON (1)
OFF (0)

different processor has diff. set of arch. of transistor
(at hardware level) $\rightarrow$ different ISA

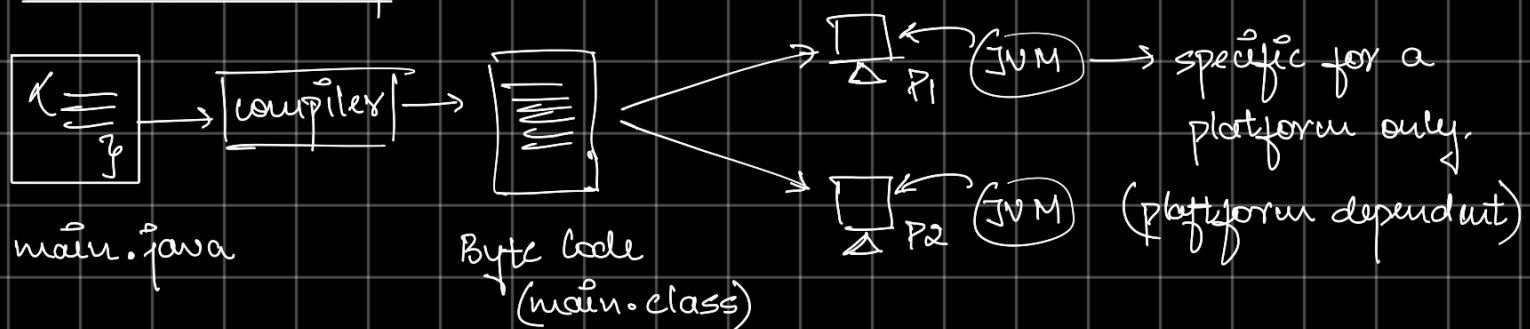


$\Rightarrow$ C / C++ problems

- platform dependent
- simplicity
- security

$\Rightarrow$ Solution by Java • portable • simple • secure

* BYTE CODE Concept



JVM $\rightarrow$ converts byte code to machine code for that platform

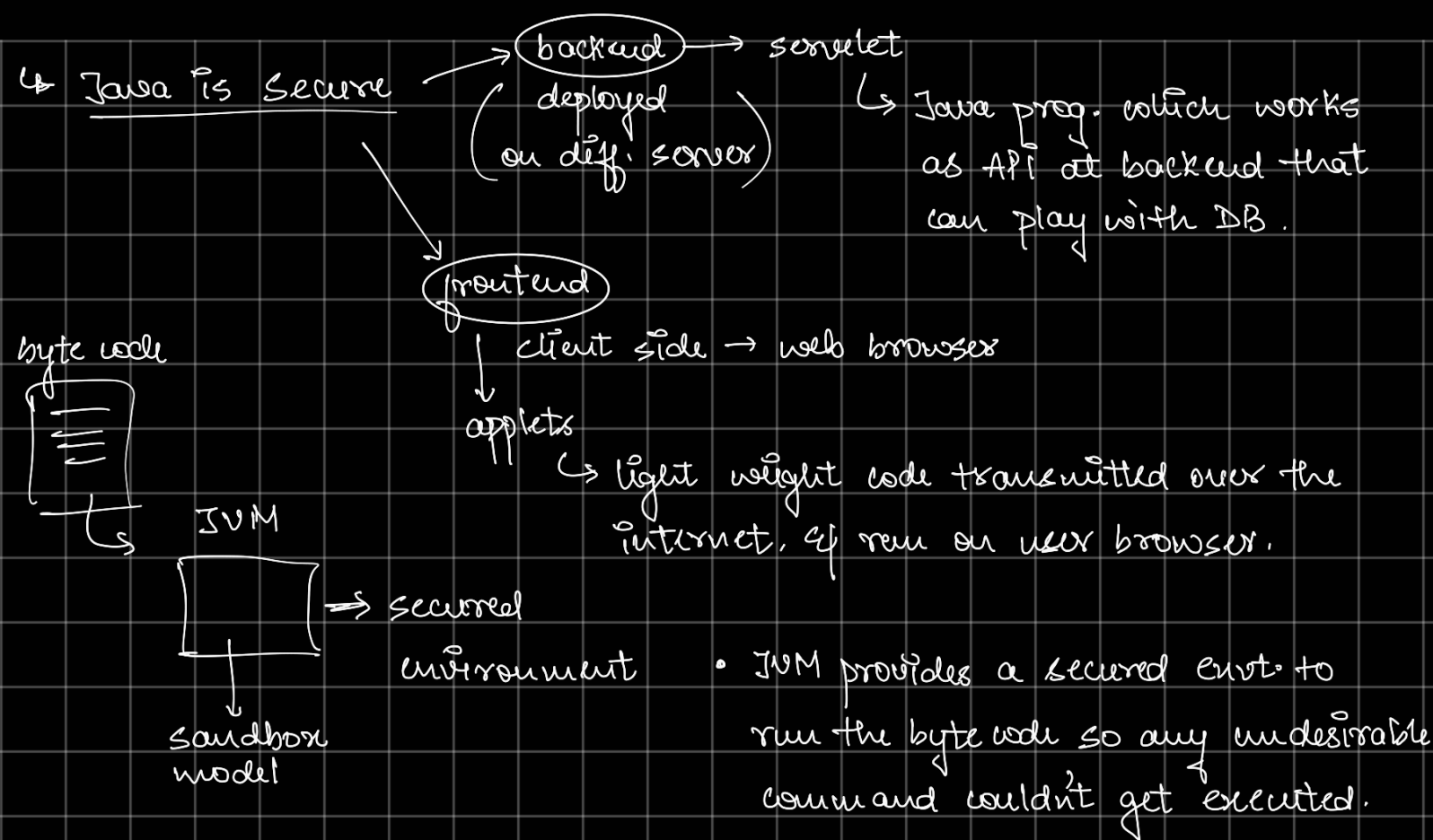
Java Virtual Machine

- mem. mgmt
- garbage collection
- security
- platform independence $\rightarrow$ (.class file can run on any platform.)

• Java is portable (platform independent)
 $\Rightarrow$ WORA (write once, run anywhere)

Java is Simple

C / C++ $\rightarrow$ pointers, multiple inheritance, manual mem. alloc. / dealloc.
 $\Rightarrow$ Java removes all these complex concepts by its syntax.



⇒ Can C/C++ be platform independent?

↳ by using bytecode + JVM

• Microsoft → C#
→ python